

A.4.4 DATA FORMATS

Table 16: Exact match of 0.1B models trained on integer addition and float addition respectively with various compositions of reverse formatting and zero padding.

	Integer Addition				Float Addition									
	rev	rev +pad	no	pad	rev total	rev total + pad	rev each	rev each + pad	rev dec	rev dec + pad	rev int	rev int + pad	no	pad
d9	0.97±.05	1.00 ±.00	0.98±.02	1.00 ±.01	0.11±.01	0.24±.00	0.12±.00	0.24±.00	0.12±.00	0.24±.00	1.00 ±.00	1.00 ±.00	0.99±.01	1.00 ±.00
d10	0.69±.11	0.91 ±.05	0.16±.11	0.50±.34	0.07±.03	0.21±.01	0.10±.02	0.23±.00	0.07±.02	0.17±.04	0.97 ±.03	0.87±.16	0.17±.04	0.76±.19

We provide the experiments in Table 16 and the evaluation curves of compositions of reverse formatting, zero padding and index hints in Figure 14, Figure 15, Figure 16, Figure 17 and Figure 18. We experiment on 0.1B models trained on 1- to 8- digit training samples. Here we all use the exact match metric.

Previous work (Zhou et al., 2024b) believes that reverse formatting can help the calculation of each digit by aligning the calculation order to the right-to-left order that humans are accustomed to and solve the carrying problem. That is, from left-to-right, we cannot determine the result of the current digit unless the next digit results and whether there is a carrying have been known. However, a more detailed analysis can explain why the order is not as important as previously believed:

Regarding addition, the cases where reverse formatting can make a difference through the effects of assisting carry-over calculations are quite rare. Most of the time, knowing the result of the next digit allows us to determine the answer for the current digit. When the next digit addition is not less than 10 (without considering further carrying from the following digit), there must be a carrying from that digit into the current one, no matter what the result of the later digits is. And when the next digit addition is not more than 8, there will never be a carrying. The only exception is the next digit addition is 9. In this situation, we must refer to the next two digits to determine the current digit results. Therefore, we point out that, although in the worst-case scenario, performing non-reversed addition requires $O(n)$ -length looking forward for each digit ($44445 + 55556 = 100001$), and reversing could solve this problem, such cases are extremely rare. In most instances, the task can be accomplished with a very limited *local view*.

About the experiments of index hint, we show in Table 17. Our conclusion on index hints seems to contradict the findings of Zhou et al. (2024b), where models with index hints appeared to achieve better results. We believe this discrepancy may be related to model size and digit range. In their work, a much smaller model (only 25M parameters) was used, but the training range covered 1-40 digits. This reduced the model’s ability to learn the patterns independently without external hints, resulting in a different learning outcome where the model began to rely on index hints. As a piece of evidence, when Zhou et al. (2024b) train 1-10 digits, the performance without index hint is OK. (But they did not provide the complete results of 1-10 digit training in their work.) The effectiveness of index hints may involve complex interactions, which could be an interesting direction for future research.

A.4.5 NUPA FINETUNING WITH PE, TOKENIZER AND REPRESENTATION MODIFICATION

We show parts of results of our attempt to finetune a Llama-3.1-8B model with PE, tokenizer and data format modification in Table 18. All the checkpoint we select by the lowest valid loss. No one can outperform the naive finetuning or the original Llama.

A.5 RULE-FOLLOWING CHAIN-OF-THOUGHT

A.5.1 DATA FORMAT OF RULE-FOLLOWING CoT

Rule-following CoT consists of a rule prompt and a response to follow the rule. To generate the CoT samples for a task, first the computational rules are converted to Python code. We utilize a code LLM to write the code in this experiment. Then, a data generation script which can generate the rule-following intermediate steps for any given input is manually written. With the script, given a pair of numbers and a task, both the rules (code prompt) and the response can be generated automatically. In the finetuning stage, both the prompt and the response are provided to the model but only the loss on the response part is calculated. During inference, the code prompt is still provided and the models are expected to generate the rule-following steps as well as the final answer.

Below is an example of a complete Rule-Following CoT data format, where the model is required to solve integer addition task with a right-to-left recursion and three digit addition as the unit task.

Prompt:

Follow the given rule to solve the question.

Rule:

```
def add(num1, num2):
    result = ''
    carry = 0
    # Main Loop
    while num1 or num2:
        digit1 = int(num1[-3:]) if num1 else 0
        digit2 = int(num2[-3:]) if num2 else 0
        total = digit1 + digit2 + carry
        result = str(total%1000) + result
        carry = total//1000
        num1 = num1[:-3] if num1 else num1
        num2 = num2[:-3] if num2 else num2
    if carry:
        result = str(carry) + result
    result = result.lstrip('0') or '0'
    return result
```

Q: Return the answer as an integer without any delimiter, like 123. Add two numbers: 28 + 35

Response:

```
num1 = "28"
num2 = "35"
result = ""
carry = 0

while num1 or num2:

    check the stop criterion
    enter the loop

    digit1 = int(num1[-3:]) if num1 else 0
    digit2 = int(num2[-3:]) if num2 else 0

    num1 = "28"
    num2 = "35"
    digit1 = 28
    digit2 = 35

    total = digit1 + digit2 + carry

    carry = 0
    total = 28 + 35 + 0 = 63

    result = str(total%1000) + result
    carry = total//1000
```

```

total % 1000 = 63 % 1000 = 63
result = "063" + "" = "063"
carry = 0 // 1000 = 0

num1 = num1[:-3]
num2 = num2[:-3]

num1 = ""
num2 = ""

while num1 or num2

check the stop criterion
num1 = "" and num2 = ""
end the loop

if carry:
    result = str(carry) + result

carry = 0
pass

result = result.lstrip("0") or "0"

result = "63"

return result

return "63"
So the answer is 63.

```

A.5.2 MAXIMUM DIGIT LENGTHS WITHIN CONTEXT WINDOW

The selective tasks used to train the RFFT are shown in Table 19 and we also report the maximal length within 2k tokens context windows limitation.